Übungsblatt 4: Aktorik & Sicherheit

Mikrocomputertechnik 3 (MCT3)

Musterlösung

Zielsetzung

Sie vertiefen Vorlesung 4: Hardware-Timer für PWM (PSC, ARR, CCR), Duty Cycle und mittlere Spannung, Software- vs. Hardware-PWM sowie Watchdog-/Window-Watchdog-Sicherheit — inklusive kurzer C-Praxis.

Bearbeitungszeit: ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 14 P, Aufgabe 2: 8 P (Gesamt: 22 P)

Teil 1: Theorie — Hardware-Counter, PWM und Watchdog

Aufgabe 1.1: Komponenten des Hardware-Timers

PWM wird in Hardware oft über General-Purpose-Timer (GPT) erzeugt.

1. Erklären Sie die Funktion von Prescaler (`PSC`), Auto-Reload Register (`ARR`) und Capture/Compare Register (`CCR`) bei der PWM-Erzeugung.
2. Erläutern Sie anhand von Zählerstand (`CNT`) und `CCR`, wie bei Edge-Aligned PWM (Active-High) HIGH und LOW entstehen.

Musterlösung Aufgabe 1.1

1. Der *Prescaler* (`PSC`) teilt den Systemtakt, damit der Counter nicht zu schnell überläuft und längere Perioden möglich sind. Das *Auto-Reload Register* (`ARR`) legt die Periodengrenze fest: erreicht `CNT` diesen Wert, setzt die Hardware typisch auf 0 zurück — damit steuert `ARR` PWM-Periode bzw. Grundfrequenz. Ob die Periode ARR oder $ARR + 1$ Ticks umfasst, steht im Datenblatt. Das *Capture/Compare Register* (`CCR`) ist die Compare-Schwelle und bestimmt das Tastverhältnis (Duty Cycle).

2. Edge-Aligned, Active-High (Up-Counting, vereinfachtes Vorlesungsmodell): Solange CNT $<$ CCR, liegt der Pin auf HIGH. Beim Compare Match (CNT erreicht CCR) zieht die Hardware auf LOW; der Pin bleibt LOW bis zum Periodenende (`ARR`), danach beginnt der Zyklus erneut (Pin wieder HIGH). Polarität und Zählmodus können im Datenblatt abweichen.

Aufgabe 1.2: Mathematik der PWM-Erzeugung

Ein SoC läuft mit $f_{\text{clock}} = 100\,\text{MHz}$. Timer: `PSC = 99`, `ARR = 1000`, `CCR = 250`. Pin: 0 V / 3,3 V.

1. Berechnen Sie die Zählfrequenz f_{cnt}. Formel angeben.
2. Bestimmen Sie den Duty Cycle in Prozent (vereinfachtes Modell CCR/ARR).
3. Nähern Sie die mittlere Spannung V_{mittel} am Pin (Vorlesungsmodell: Mittelwert $\approx V_{\text{Pin}} \times$ Duty Cycle).

Musterlösung Aufgabe 1.2

1. $$f_{\text{cnt}} = \frac{f_{\text{clock}}}{\text{PSC} + 1} = \frac{100\,\text{MHz}}{99 + 1} = 1\,\text{MHz}$$

2. Vereinfachtes Vorlesungsmodell (CCR/ARR, Periode analog $0 \ldots 999$ bei ARR = 1000):

$$\text{Duty Cycle} = \frac{\text{CCR}}{\text{ARR}} = \frac{250}{1000} = 0{,}25 = 25\,\%$$

3. $$V_{\text{mittel}} \approx V_{\text{Pin}} \times \text{Duty Cycle} \approx 3{,}3\,\text{V} \times 0{,}25 = 0{,}825\,\text{V}$$

(Hinweis: Manche Timer zählen $ARR+1$ Schritte pro Periode — dann Duty ggf. CCR/(ARR+1). Immer Datenblatt prüfen. Die Vorlesungsgröße V_{mittel} meint hier den zeitlichen Mittelwert, nicht den RMS-Wert $\propto \sqrt{\text{Duty}}$.)

Aufgabe 1.3: Software-PWM vs. Hardware-PWM

1. Erklären Sie Bit-Banging (Software-PWM) und nennen Sie zwei Nachteile gegenüber Hardware-PWM.

Musterlösung Aufgabe 1.3

Beim *Bit-Banging* (Software-PWM) toggelt die CPU den Pin per Software (z. B. mit `delay`-Schleifen) zwischen HIGH und LOW.

Nachteile gegenüber Hardware-PWM:

1. Die CPU steckt während der Wartezeiten im Busy-Waiting und ist für andere Aufgaben blockiert.
2. Interrupts unterbrechen die Delays → Jitter; Flanken wandern (Motor zuckt, LED flackert).

Aufgabe 1.4: Watchdog Timer (WDT)

1. Was ist ein Watchdog Timer, und was passiert hardwareseitig bei Zählerstand 0?
2. Warum ist das Füttern des WDT in einer periodischen Timer-ISR (z. B. SysTick) gefährlich? Wo sollte `wdt_feed()` stattdessen sitzen?
3. Welches zusätzliche Problem adressiert ein Window Watchdog (WWDG)?

Musterlösung Aufgabe 1.4

1. Der Watchdog Timer (WDT) ist ein vom Programmfluss/CPU-Kern unabhängiger Hardware-Zähler (meist Down-Counter). Erreicht er 0 (Timeout), greift er in das Power-/Reset-Management ein und löst typisch einen harten System-Reset aus — das System bootet neu.

2. Füttert man den WDT in einer periodischen Timer-ISR (z. B. SysTick), kann die Hauptschleife in einem Deadlock stecken, während die ISR weiterläuft und den Hund unbemerkt füttert — der Freeze bleibt unerkannt. `wdt_feed()` gehört daher in die unterste Ebene der `while(1)`-Geschäftslogik (Main-Loop), als Beweis, dass die Anwendungslogik noch fortschreitet.

3. Ein Window Watchdog (WWDG) erkennt zusätzlich zu spätes Timeout *und* zu frühes/zu häufiges Füttern (z. B. fehlerhafte Endlosschleife mit `wdt_feed()`): Feeds sind nur in einem strikten Zeitfenster erlaubt.

Teil 2: Praxis — PWM und Watchdog in C

Adressen sind didaktisch/fiktiv.

Aufgabe 2.1: LED-Helligkeit per Hardware-PWM

`TIMER_ARR = 1000`. Helligkeit als Prozent 0…100.

```c
#include <stdint.h>

#define TIMER_BASE 0x40000000u
#define TIMER_ARR  (*(volatile uint32_t *)(TIMER_BASE + 0x04u))
#define TIMER_CCR1 (*(volatile uint32_t *)(TIMER_BASE + 0x0Cu))

void set_led_brightness(uint8_t percent) {
    /* 1. percent auf max. 100 begrenzen */
    /* TODO */

    /* 2. CCR aus Prozent und TIMER_ARR berechnen */
    /* TODO */

    /* 3. in TIMER_CCR1 schreiben */
    /* TODO */
}
```

Musterlösung Aufgabe 2.1

```c
#include <stdint.h>

#define TIMER_BASE 0x40000000u
#define TIMER_ARR  (*(volatile uint32_t *)(TIMER_BASE + 0x04u))
#define TIMER_CCR1 (*(volatile uint32_t *)(TIMER_BASE + 0x0Cu))

void set_led_brightness(uint8_t percent) {
    if (percent > 100u)
        percent = 100u;

    uint32_t ccr_val = ((uint32_t)percent * TIMER_ARR) / 100u;
    TIMER_CCR1 = ccr_val;
}
```

Aufgabe 2.2: Freeze provozieren und WDT-Heilung

Simulieren Sie einen Deadlock per Tastendruck. Nach WDT-Reset soll die Ursache auf der Konsole erscheinen. Füttern Sie den WDT an der architektonisch richtigen Stelle.

```c
#include <stdio.h>
#include <stdint.h>

#define RESET_REASON_REG (*(volatile uint32_t *)0x40020000u)
#define WDT_RESET_FLAG   (1u << 2)

void check_reset_reason(void) {
    /* TODO: WDT_RESET_FLAG prüfen und ggf. warnen */
}

int main(void) {
    check_reset_reason();
    init_pwm();
    init_wdt(); /* Timeout z. B. 2 s */

    while (1) {
        if (button_pressed()) {
            printf("Freeze provoziert! Warte auf WDT...\n");
            /* TODO: künstlicher Deadlock */
        }

        set_led_brightness(50);
        /* TODO: wdt_feed() an korrekter Stelle */
    }
}
```

Musterlösung Aufgabe 2.2

```c
#include <stdio.h>
#include <stdint.h>

#define RESET_REASON_REG (*(volatile uint32_t *)0x40020000u)
#define WDT_RESET_FLAG   (1u << 2)

extern void init_pwm(void);
extern void init_wdt(void);
extern int button_pressed(void);
extern void set_led_brightness(uint8_t percent);
extern void wdt_feed(void);

void check_reset_reason(void) {
    if (RESET_REASON_REG & WDT_RESET_FLAG)
        printf("WARNUNG: WDT-Reset nach Freeze erkannt!\n");
}

int main(void) {
    check_reset_reason();
```

```
    init_pwm();
    init_wdt();

    while (1) {
        if (button_pressed()) {
            printf("Freeze provoziert! Warte auf WDT...\n");
            while (1) { /* Deadlock: kein wdt_feed() */ }
        }

        set_led_brightness(50);
        wdt_feed(); /* in der Main-Loop, nicht in einer Timer-ISR */
    }
}
```